



Norwegian University of
Science and Technology

Formally Verified Reduction Proof for the Half-Gates Garbling Scheme

Prateek Sharma

Submission date: May 2026
Main supervisor: Tjerand Silde, NTNU
Co-supervisors: Russel W. F. Lai and
Chris Brzuska, Aalto University

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Garbled circuits are a fundamental cryptographic primitive used in secure two-party computation (2PC), enabling two parties to jointly compute a function without revealing their private inputs—only the output is revealed. The concept was introduced by Andrew Yao in 1986 [Yao86], where functions are expressed as Boolean circuits and each wire is labelled with encrypted wire keys. Early constructions required four ciphertexts per gate, resulting in significant communication overhead and limiting practical adoption.

Subsequent optimizations such as Point-and-Permute and the Half-Gates technique greatly reduced the number of ciphertexts needed per gate, leading to substantial performance improvements [BMR90; ZRE15]. These optimizations form the basis of many state-of-the-art secure computation frameworks.

Formal verification of cryptographic constructions is essential to ensure correctness and prevent subtle vulnerabilities arising from unchecked corner cases. Brzuska and Oechsner [BO23] developed a state-separating pen-and-paper reduction proof for Yao’s original garbling scheme. State-separating proofs model protocol components as isolated “packages” with private state and oracle interfaces, allowing for structured security reductions. Their work established security by reducing the garbling scheme to the IND-CPA security of the encryption primitive used for gate tables.

Domino is an automated verification tool specifically designed for state-separating proofs. It has been used to reason about complex real-world protocols such as TLS [Raj25] and WPA2 [BDE+22]. Domino offers a high level of automation and readability because proof scripts resemble the pseudocode found in traditional cryptographic proofs.

Furthermore, the above mentioned proof for Yao’s garbling scheme by Brzuska and Oechsner [BO23] was later formalized in Domino [BW]. It can serve as a good starting point for the thesis to observe how games may be converted from pen and paper proofs to Domino code.

The goal of this thesis is to formalize and mechanize a security reduction for the Half-Gates garbling scheme within the Domino framework. This includes:

- Developing a state-separating, pen-and-paper reduction proof for the Half-Gates technique.
- Encoding this proof in Domino.

This work extends formal verification techniques to an optimized and widely deployed garbling construction and demonstrates Domino’s applicability to modern cryptographic primitives.

Approved: yyyy-mm-dd – Tjerand Silde, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of

the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures	vii
List of Tables	ix
List of Algorithms	xi
1 Example	1
1.1 First section (middling version)(ToC)	2
1.1.1 First subsection with some <i>Math</i> symbol	2
1.1.2 Mathematics	2
1.1.3 Source code example	3
2 Introduction	5
3 Background	7
3.1 Multi-Party Computation (MPC)	7
3.2 Garbling Schemes	7
3.2.1 Oblivious Transfer Protocols	7
3.3 Optimizations to Yao's Garbling scheme	7
3.3.1 Point and Permute	7
3.3.2 Free XOR	8
3.3.3 Garbled Row Reduction - 3	8
3.3.4 Garbled Row Reduction - 2	8
3.3.5 FleXOR	8
3.3.6 Half-Gates	8
3.3.7 Garbled Gadgets	8
3.4 Formal Verification and Tools	8
3.4.1 Easycrypt	8
3.4.2 Domino	8
4 Pen-and-Paper State-Separating Proof	9
4.1 State Separating Proofs	9

4.2	Existing Half-Gates Proof and how it is to be modelled into state separating proofs	9
5	Domino	11
5.1	What is Domino?	11
5.2	Using Domino	11
5.2.1	Dependencies	11
5.2.2	Project Structure	12
5.3	Coding Half-gates in Domino	12
5.4	Improvements	12
6	Conclusion	13
	References	15

List of Figures

1.1	A figure	2
-----	--------------------	---

List of Tables

1.1	A table	3
-----	-------------------	---

List of Algorithms

1.1	The Hello World! program in Java.	3
-----	---	---

Chapter 1

Example

Here is an example of how to use acronyms such as Norwegian University of Science and Technology (NTNU). The second time only NTNU is shown and if there were several you would write NTNUs. And here is an example¹ of citation [NNNN21].

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look.

¹A footnote

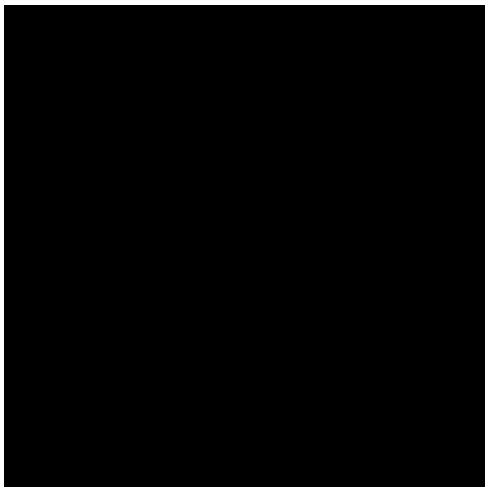


Figure 1.1: A figure

This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

1.1 First section (verbose version)

1.1.1 First subsection with some *Math* symbol

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

- item1
- item2
- ...

1.1.2 Mathematics

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. $\sin^2(\alpha) + \cos^2(\beta) = 1$. If you read this text, you will get no information $E = mc^2$. Really? Is there no information? Is there a

Table 1.1: A table

a	b	c	d	e
f	g	h	i	j
k	l	m	n	o
p	q	r	s	t
u	v	w	x	y
z	æ	ø	å	

difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. $\sqrt[n]{a} \cdot \sqrt[n]{b} = \sqrt[n]{ab}$. This text should contain all letters of the alphabet and it should be written in of the original language. $\frac{\sqrt[n]{a}}{\sqrt[n]{b}} = \sqrt[n]{\frac{a}{b}}$. There is no need for special content, but the length of words should match the language. $a \sqrt[n]{b} = \sqrt[n]{a^n b}$.

Proposition 1.1. *A proposition... (similar environments include: theorem, corollary, conjecture, lemma)*

Proof. And its proof. □

1.1.3 Source code example

Algorithm 1.1 The Hello World! program in Java.

```
class HelloWorldApp {
    public static void main(String[] args) {
        //Display the string
        System.out.println("Hello World!");
    }
}
```

You can refer to figures using the predefined command like Figure 1.1, to pages like Page 1.1, to tables like Table 1.1, to chapters like Chapter 1 and to sections like Section 1.1 and you may define similar commands to refer to proposition, algorithms etc.

Chapter 2

Introduction

The introduction must contain the following topics at the least:

- Paragraph: MPC and significance
- Garbling schemes and optimizations
- Formal Verification
- WIP TOOL Domino
- What does the thesis do? Use domino to formally verify a proof for Half Gates

Multiparty computation (MPC) is a common scenario in the present time, where data is transferred and operated on across multiple devices in order to provide services. A very obvious example is deferring some computation to a cloud server which has higher and more elastic computation capabilities. As a consequence, there also exist regulations as well as requirements that bring the need to secure such data from third parties, such as in the case of healthcare, or personal information (cite_gdpr). Two-party computation (2PC) is a subset of MPC in which two parties want to evaluate a function that takes inputs from both parties, but with the constraint that they do not wish to reveal the inputs to each other. Examples could be matching if a user's password is in a leak database, or jointly computed biometric validation, which are actually examples of a more well defined problem called Private Set Intersection (PSI) [PSZ14].

Chapter 3

Background

The following sections can be split into their own chapters (if needed).

3.1 Multi-Party Computation (MPC)

3.2 Garbling Schemes

How do these relate to multi party Computation

Garbling as a cryptographic goal, as opposed to a technique to achieve some specific properties.

The properties that circuit garbling provides - privacy, authenticity and obliviousness. General definition of a garbling scheme: [BHR12]. The authors use notions of strings and functions being separate so that the garbling scheme remains generalizable and not specific to garbling boolean circuits or arithmetic circuits. Garbling scheme maps a function f to an encoding function En , a decoding function De , and an evaluation function Ev , such that when an input x is encoded by En , then the encoded value is evaluated through Ev , and the resulting output is decoded by De , the resultant value is the same as the input being evaluated by f .

Security notions definition may be included.

3.2.1 Oblivious Transfer Protocols

Why are they used in garbling schemes. This can be reduced to one line or something. Its not important to Half-Gates of Yao SSP.

3.3 Optimizations to Yao's Garbling scheme

3.3.1 Point and Permute

How many bits are supposed to be used for the wire "color"? If (x,y) are all the inputs to the circuit (if there are more than 2 inputs, x is $x_1x_2x_3...$ and similarly for

y, i.e., for each binary input label x and y , $\text{len}(x) = 2^n$ where n is the number of x inputs. similarly for y). So X and Y are the inputs domain for the two parties, and there need to be $\lceil \log(X) \rceil$ color bits after the key to preserve key length to be equal to the security parameter, while still having the ability to point to a specific X bit on the garbled truth table [EKR18].

3.3.2 Free XOR

3.3.3 Garbled Row Reduction - 3

Compatible with Free XOR

3.3.4 Garbled Row Reduction - 2

3.3.5 FleXOR

3.3.6 Half-Gates

End with why we chose this one to work one

3.3.7 Garbled Gadgets

3.4 Formal Verification and Tools

Start from Formal verification in general (read more too), and then go into tools used for formal verification in cryptography.

3.4.1 Easycrypt

3.4.2 Domino

How is it different from easycrypt? why did we choose this? what is a state separating proof and why is this more suited to those?

Chapter 4

Pen-and-Paper State-Separating Proof

Read half gates and chris's paper to understand what to do here even. (WIP)

4.1 State Separating Proofs

- What are games and packages
- Something about why state separating proofs exist. "composable" and "constructive" security
- State separating proofs allow us to make intermediate arguments (hybrids) in a reusable manner. It is often more intuitive to model something as its own package that has inputs and outputs than to have it be part of many different codes and realise that its the same thing.

Composability of layers is very important so we can derive circuit security from just proving layer security. The simulator for layer n must output a garbling of its output which is then supplied to a new simulator for layer $n+1$. Therefore, if a simulator can simulate a gate by getting its inputs (garbled input) and return garbled output, it can be composed with the simulator for the next gate and so on. Same idea for layers.

Actually, to add to the above, the simulator does NOT know the input. It only gets the circuit information loss of the garbled circuit $\Phi(C)$, the output of the garbled circuit Y , and the security parameter 1^λ . So it samples a random input label (in this case its the label, in the original Yao paper it was a random bitstring sampled for the wire key, but since we are working in a setting where the FreeXOR optimization is assumed, we will probably end up sampling a random Δ and sharing that state across simulator invocations to generate valid labels for the wires.)

4.2 Existing Half-Gates Proof and how it is to be modelled into state separating proofs

Chapter 5

Domino

5.1 What is Domino?

What is domino and how does it work? The layers that exist, and the design in general.

Domino is a proof assistant for State-separating proofs (SSPs) that allows the user to validate a SSP using familiar syntax, and an underlying SMT solver. It uses very similar syntax and constructs as SSPs, such as packages, games and theorems. The explicit relation between these games is defined using invariant files, written in the SMT-LIB language. These games are then checked against different lemmata that are needed to fulfil the relation that the user wants to establish.

Domino is published as a rust package available on Github. The domino program/crate itself contains a compiler that has its own defined language that intentionally tries to be very similar the SSP syntax defined in [BDF+18].

5.2 Using Domino

5.2.1 Dependencies

Since Domino is published as a Rust package, it requires a Rust toolchain, which can be installed using *rustup*. Rustup manages multiple Rust toolchains, and is the most straightforward way to getting set up on Linux. Rustup is typically installed using a bash script from the official website, but it is also available on most system package managers. It can also be built from source. Installing rustup automatically installs the Rust binary and cargo (the package manager), among other components. These can also be obtained without rustup, and the installation page explains the methods in more detail [Rus].

The second prerequisite to run Domino is a functioning CVC5 installation, and for the cvc5 executable to be added to PATH, so that Domino can invoke it while

running. CVC5 can be built from source following the instructions in [CVC]. CVC5 is the SMT solver that checks whether the constraints given by our proof are satisfiable or not. This is done in practice by taking the negation of the condition we want, and checking for satisfiability of that negation. If it exists, the condition is false.

5.2.2 Project Structure

A Domino project directory requires the presence of an empty *ssp.toml* file. It allows SSP logic to be written into packages, games and theorem files. These are all placed under their own directories, i.e., folders called *packages*, *games*, and *theorem*.

Each of these folders can then contain one or more files. Domino checks packages and games for syntactic correctness even if they are not used in a theorem. This is intentional design, since when writing state separating proofs, the components (packages and games) are usually set up before the actual proof.

In older versions of Domino, theorems were called proofs, and the folder and file header for proofs were named as such.

An Example Theorem in Domino

mIND-CPA game [here](#), talk about its packages, and the two games with diagrams, and how the theorem to prove states that the two games are indistinguishable.

5.3 Coding Half-gates in Domino

5.4 Improvements

Chapter 6

Conclusion

References

- [BDE+22] C. Brzuska, A. Delignat-Lavaud, et al., “Key-schedule security for the TLS 1.3 standard”, in *Advances in Cryptology – ASIACRYPT 2022, Part I*, (Taipei, Taiwan, Dec. 5–9, 2022), S. Agrawal and D. Lin, Eds., ser. Lecture Notes in Computer Science, vol. 13791, Springer, Cham, Switzerland, 2022, pp. 621–650.
- [BDF+18] C. Brzuska, A. Delignat-Lavaud, et al., *State separation for code-based game-playing proofs*, Publication info: A major revision of an IACR publication in ASIACRYPT 2018, 2018. last visited: Apr. 10, 2026. [Online]. Available: <https://eprint.iacr.org/2018/306>
- [BHR12] M. Bellare, V. T. Hoang, and P. Rogaway, *Foundations of garbled circuits*, Publication info: Published elsewhere. Proceeding version appears in CCS 2012, 2012. last visited: Jan. 14, 2026. [Online]. Available: <https://eprint.iacr.org/2012/265>
- [BMR90] D. Beaver, S. Micali, and P. Rogaway, “The round complexity of secure protocols (extended abstract)”, in *22nd Annual ACM Symposium on Theory of Computing*, (Baltimore, MD, USA, May 14–16, 1990), ACM Press, 1990, pp. 503–513.
- [BO23] C. Brzuska and S. Oechsner, “A state-separating proof for Yao’s garbling scheme”, in *CSF 2023: IEEE 36th Computer Security Foundations Symposium*, (Dubrovnik, Croatia, Jul. 10–14, 2023), IEEE Computer Society Press, 2023, pp. 137–152.
- [BW] R. Brzuska Egger and Winkermann, private communication.
- [CVC] CVC5. “Installation — cvc5 documentation”, last visited: May 25, 2026. [Online]. Available: <https://cvc5.github.io/docs/cvc5-1.0.0/installation/installation.html>
- [EKR18] D. Evans, V. Kolesnikov, and M. Rosulek, “A Pragmatic Introduction to Secure Multi-Party Computation”, en, *Foundations and Trends® in Privacy and Security*, vol. 2, no. 2-3, p. 34, Dec. 2018. last visited: May 27, 2026. [Online]. Available: <https://www.emerald.com/ftsec/article/2/2-3/70/1331393/A-Pragmatic-Introduction-to-Secure-Multi-Party>
- [NNNN21] F. Name1, F. Name2, et al., “A dummy title”, *A Fake Journal*, vol. 1, no. 1, pp. 000–000, Jun. 2021.

- [PSZ14] B. Pinkas, T. Schneider, and M. Zohner, “Faster private set intersection based on OT extension”, in *USENIX Security 2014: 23rd USENIX Security Symposium*, K. Fu and J. Jung, Eds., San Diego, CA, USA: USENIX Association, Aug. 20–22, 2014, pp. 797–812. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/pinkas>
- [Raj25] A. Rajabi, “Towards formally verifying the TLS 1.3 Key Schedule Security in SSBe”, en, *Master thesis, Aalto University, 2025*, Jul. 2025. [Online]. Available: <https://aaltodoc.aalto.fi/handle/123456789/138153>
- [Rus] Rust Foundation. “Other Installation Methods - Rust Forge”, last visited: May 25, 2026. [Online]. Available: <https://forge.rust-lang.org/infra/other-installation-methods.html>
- [Yao86] A. C.-C. Yao, “How to generate and exchange secrets (extended abstract)”, in *27th Annual Symposium on Foundations of Computer Science*, (Toronto, Ontario, Canada, Oct. 27–29, 1986), IEEE Computer Society Press, 1986, pp. 162–167.
- [ZRE15] S. Zahur, M. Rosulek, and D. Evans, “Two halves make a whole - reducing data transfer in garbled circuits using half gates”, in *Advances in Cryptology – EUROCRYPT 2015, Part II*, (Sofia, Bulgaria, Apr. 26–30, 2015), E. Oswald and M. Fischlin, Eds., ser. Lecture Notes in Computer Science, vol. 9057, Springer Berlin Heidelberg, Germany, 2015, pp. 220–250.